

可信开源软件大作业报告

基于多维数据的软件仓库分析工具 设计与实现

Design and Implementation of Multi-dimensional
Software Repository Analysis Tool

学院 (学部): 软件学院
专 业: 软件工程
学 生 姓 名: 开发者
学 号: 20240001
指 导 教 师: 指导教师 教授
评 阅 教 师: 评阅教师
完 成 日 期: 2026 年 2 月

大连理工大学

Dalian University of Technology

目 录

引 言	1
1 git 仓库分析	2
1.1 基于 gitgraph 的 git 提交历史可视化	2
1.2 多维度 git 仓库分析	2
1.2.1 实现技术与工具	2
1.2.2 分析维度详解	2
1.2.2.1 开发活跃度分析	2
1.2.2.2 贡献者画像分析	3
1.2.2.3 代码演进与质量分析	4
1.2.2.4 提交行为语义分析	5
2 代码静态/动态分析	7
2.1 代码复杂度分析	7
2.1.1 分析方法与实现	7
2.1.2 可视化分析	7
2.2 项目依赖与安全审计	7
2.2.1 分析维度	7
2.2.2 结果展示	8
2.3 代码漏洞扫描 (SAST)	8
2.3.1 扫描机制	8
2.3.2 风险评估	9
2.4 代码质量与动态分析	9
3 github 仓库分析	12
3.1 GitHub PR (Pull Request) 分析	12
3.1.1 生命周期与状态统计	12
3.1.2 僵尸 PR (Zombie PRs) 识别	12
3.1.3 审查效率评估	12
3.2 GitHub Issue 安全分析	12
3.2.1 安全关键词挖掘	12
3.2.2 趋势与分类统计	12

3.3 GitHub Actions 工作流分析	13
3.3.1 工作流安全性检测	13
3.3.2 运行效率分析	15
结 论	16
附录 A 附录内容名称	17

引 言

从三个维度分析：git 仓库分析，代码动态/静态分析，github 仓库分析

1 git 仓库分析

1.1 基于 gitgraph 的 git 提交历史可视化

本分析工具包含一个 `html_generator.py` 模块，用于生成 Git 提交历史的可视化报告。该模块通过读取 Git 仓库的提交日志（包含哈希值、作者、日期、提交信息及父提交关系），并将这些数据注入到 HTML 模板中。

前端可视化采用了 `gitgraph.js` 开源库。该库能够以美观的图形方式展示 Git 的分支结构、合并历史以及提交节点。生成的 `git_tree.html` 文件支持交互式浏览，用户可以直观地查看项目的演进路线、分支合并情况以及各开发者的提交记录。

1.2 多维度 git 仓库分析

本节详细介绍 `analyze.py` 模块所实现的多维度 Git 仓库分析功能。该模块旨在通过量化数据揭示项目的开发节奏、团队结构及代码演进规律。

1.2.1 实现技术与工具

分析工具主要基于 Python 语言开发，核心利用了以下开源库：

- **GitPython**：用于与本地 Git 仓库进行交互。通过 `'git.Repo'` 对象，工具能够遍历提交历史（`'iter_commits'`），提取包括提交哈希、作者、时间戳、提交信息及文件变更统计（`'commit.stats'`）在内的元数据。
- **Matplotlib**：作为主要的可视化引擎，用于绘制柱状图、折线图、饼图及直方图。工具针对中文字符进行了专门配置（如加载 `'SimHei'` 或 `'Source Han Sans CN'` 字体），确保生成的图表在各种系统环境中均能正确显示中文标签。
- **Pandas**：用于高效的数据处理与分析，特别是在进行时间序列分析（如 PR 处理周期）时，利用 `DataFrame` 进行数据的聚合、重采样与统计。
- **Collections (Counter)**：Python 内置的高效计数器，用于对作者提交次数、关键词频率等数据进行快速聚合。

1.2.2 分析维度详解

1.2.2.1 开发活跃度分析

开发活跃度是衡量项目生命周期的重要指标。工具从“宏观月度趋势”和“微观时段分布”两个层面进行了分析。

月度活跃度趋势：通过提取每条提交记录的日期并截取至月份（YYYY-MM），统计每月的提交总数。如图 1.1 所示，该分析能够清晰地反映出项目的起步期、爆发期及平

稳维护期，帮助识别特定的开发冲刺活动。

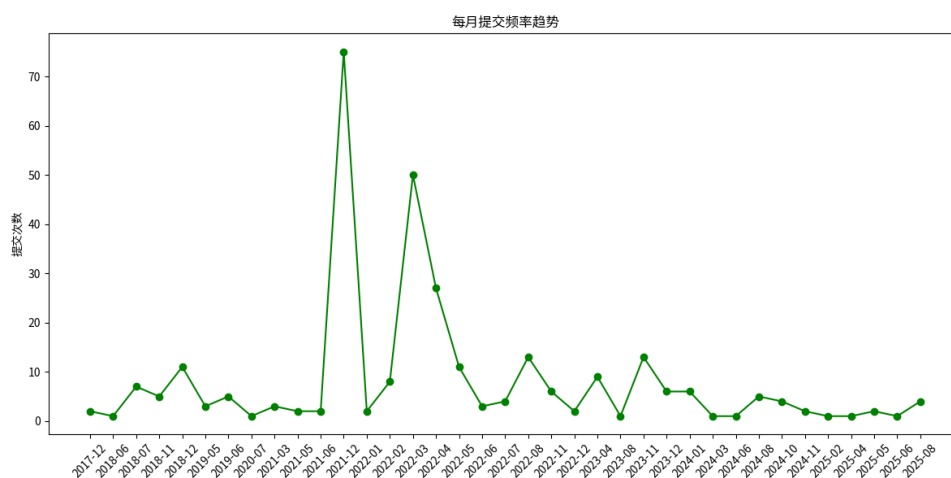


图 1.1 项目月度提交活跃度趋势

全天时段提交分布：该分析通过提取提交时间的小时属性（0-23h），绘制了开发者的工作习惯图谱。如图 1.2 所示，这不仅能反映出主力团队的工作时区，还能揭示是否存在高强度的加班或夜间突击开发情况。

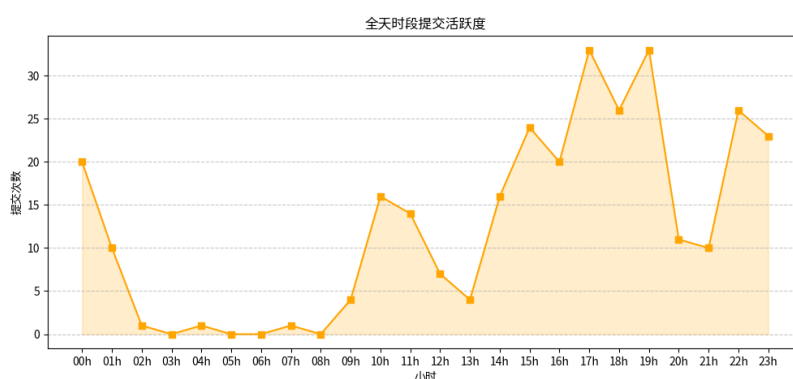


图 1.2 全天各时段提交频率分布

1.2.2.2 贡献者画像分析

为了了解团队结构，工具对 Git 提交中的 ‘author’ 字段进行了清洗和统计。

核心贡献者统计：利用 ‘Counter’ 统计各作者的提交次数，并截取前 10 名生成条形图（图 1.3）。这有助于识别项目的“核心维护者”以及社区参与的广度。在多人协作项目中，通常遵循“二八定律”，即大部分代码由少数核心成员提交。

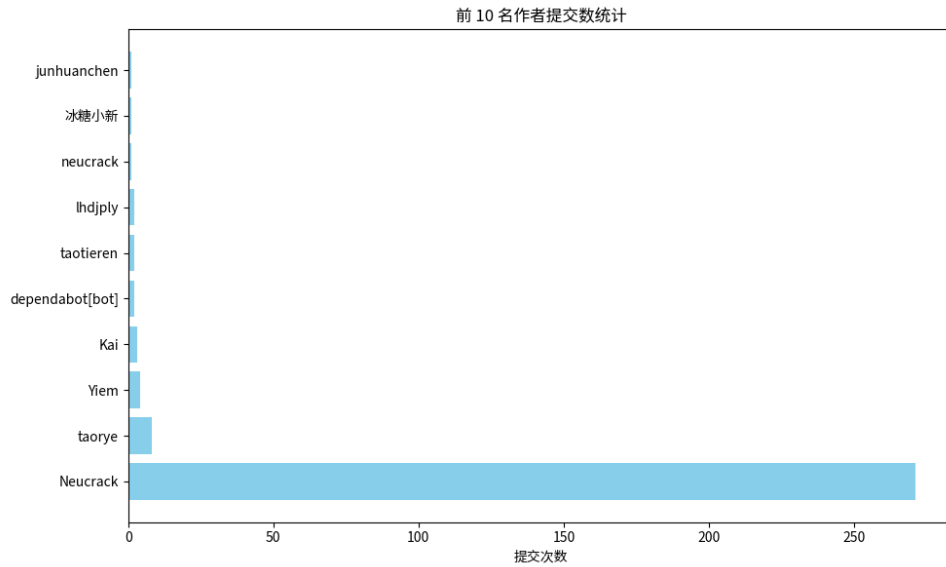


图 1.3 前 10 名作者提交贡献统计

1.2.2.3 代码演进与质量分析

代码量的变化直接反映了软件的成长过程。

代码行数 (LOC) 演变：通过累加每次提交的 ‘insertions’（新增行数）和 ‘deletions’（删除行数），计算出项目净代码行数的随时间变化的累计曲线（图 1.4）。陡峭的上升通常对应新功能的引入，而平缓或下降则可能意味着重构或死代码清理。

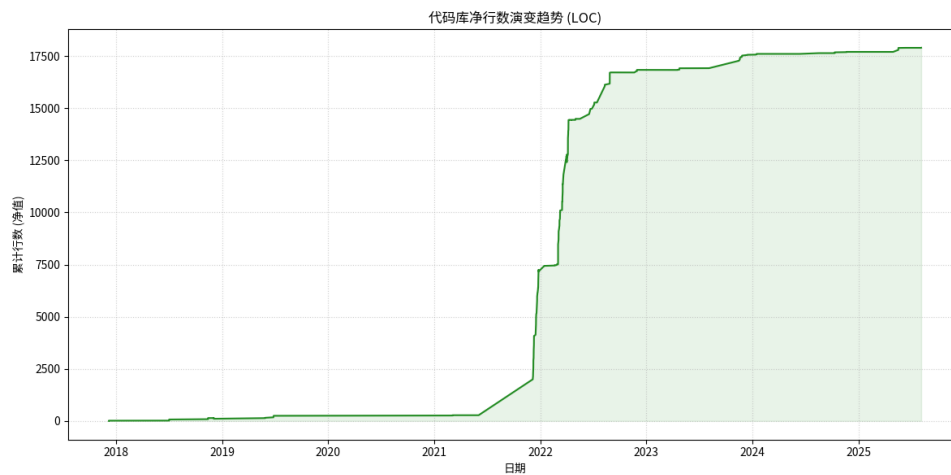


图 1.4 代码库净行数演变趋势

此外，工具还分析了**代码增删趋势**（图 1.5），分别展示了新增和删除的波动情况。如果某一时间段内“删除”曲线显著升高，通常标志着重大的架构调整或技术债务偿还。

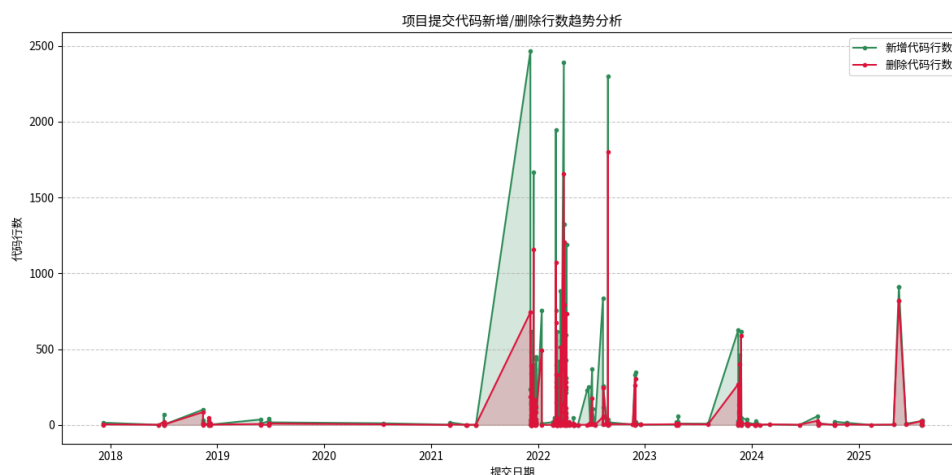


图 1.5 代码新增与删除行数趋势对照

1.2.2.4 提交行为语义分析

为了理解开发者的主要工作内容，工具对 Commit Message 进行了文本挖掘。

关键词分布分析：针对中文开发环境，工具定义了一组特定的关键词映射（如”新增/Add”、”修复/Fix”、”优化/Refactor”）。通过对提交信息进行模式匹配，生成了关键词分布饼图（图 1.6）。这一比例直观地展示了项目是处于快速功能迭代阶段（“新增”占比较高）还是稳定维护阶段（“修复”占比较高）。

修改热点分析：通过统计每次提交中涉及的文件路径，工具识别出了修改频率最高的目录或模块（图 1.7）。这些“热点”区域往往是业务逻辑最复杂、变更最频繁的部分，也是潜在 Bug 的高发区，需要重点关注。

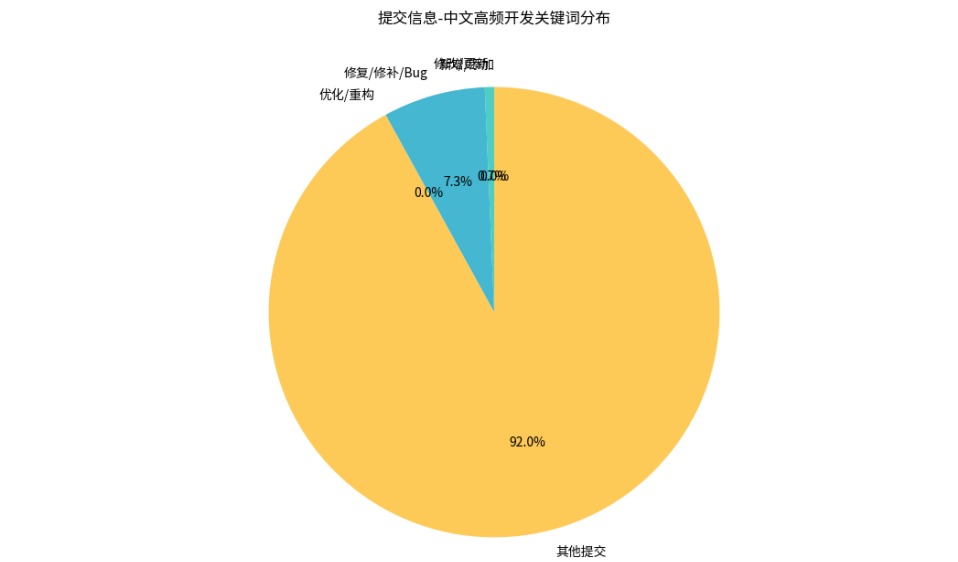


图 1.6 提交信息高频关键词分布

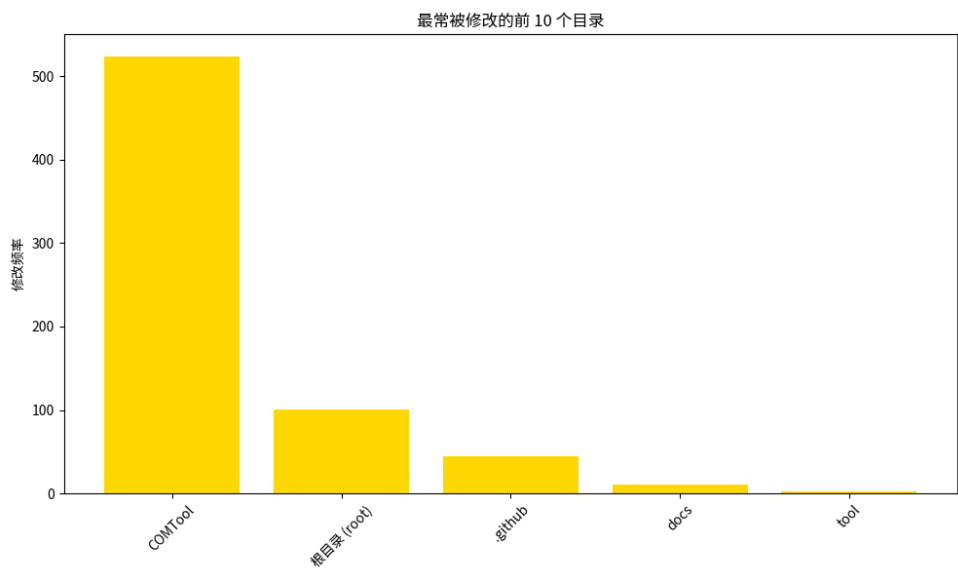


图 1.7 最常被修改的代码目录 (热点分析)

2 代码静态/动态分析

本章详细介绍针对代码库进行的深度质量分析。分析工作涵盖了代码复杂度评估、项目依赖安全审计、静态漏洞扫描以及动态测试覆盖率分析四个方面，旨在全方位评估软件的鲁棒性与可维护性。

2.1 代码复杂度分析

代码复杂度是衡量软件可维护性的重要指标。本项目利用了 `radon` 库，通过 `code_analyzer.py` 模块实现了自动化的复杂度计算与可视化。

2.1.1 分析方法与实现

模块遍历仓库中的所有 Python 文件，基于抽象语法树 (AST) 计算以下核心指标：

- **圈复杂度 (Cyclomatic Complexity, CC)**: 通过计算代码中线性独立路径的数量来衡量逻辑复杂性。`radon.complexity.cc_visit` 函数被用于解析代码块，统计控制流结构（如 `if`, `while`, `for`）。
- **可维护性指数 (Maintainability Index, MI)**: 利用 `radon.metrics.mi_visit` 计算的一个综合评分 (0-100)，该指标结合了圈复杂度、代码行数 (LOC)、Halstead 体积和注释比例。

2.1.2 可视化分析

分析结果被聚合后，工具生成了图 2.1 所示的分布图。图中上部展示了有效文件的平均圈复杂度，下部展示了可维护性指数。这能帮助开发者快速定位那些复杂度过高（CC 高）或难以维护（MI 低）的“坏味道”模块。

2.2 项目依赖与安全审计

针对现代软件开发中普遍存在的“供应链安全”问题，`dependency_analyzer.py` 模块基于 `pipdeptree` 和 `safety` 工具实现了深度的依赖分析。

2.2.1 分析维度

- **依赖树解析**: 通过调用 `pipdeptree --json` 命令，工具解析当前环境的依赖关系，构建出完整的依赖树。这不仅识别了直接依赖，还挖掘了隐蔽的传递依赖 (Transitive Dependencies)。
- **漏洞数据库比对**: 集成 `safety` 工具，将项目依赖版本与已知的 CVE 漏洞数据库进行比对，自动标记存在安全风险的包版本。



图 2.1 代码圈复杂度与可维护性指数分布

2.2.2 结果展示

图 2.2 展示了分析结果。左侧饼图反映了项目依赖的构成比例（直接依赖 vs 传递依赖），右侧直方图则统计了不同风险等级的漏洞数量。

2.3 代码漏洞扫描 (SAST)

为了在代码层面发现潜在的安全隐患，项目集成了 bandit 做为静态应用程序安全测试 (SAST) 工具。

2.3.1 扫描机制

vulnerability_scanner.py 模块封装了 BanditManager。它不执行代码，而是将 Python 源码解析为 AST，并在节点上运行一系列插件来匹配已知的漏洞模式，例如：

- 硬编码的敏感信息（密码、API Key）。
- 弱加密算法（如 MD5, SHA1）的使用。

项目依赖分析报告

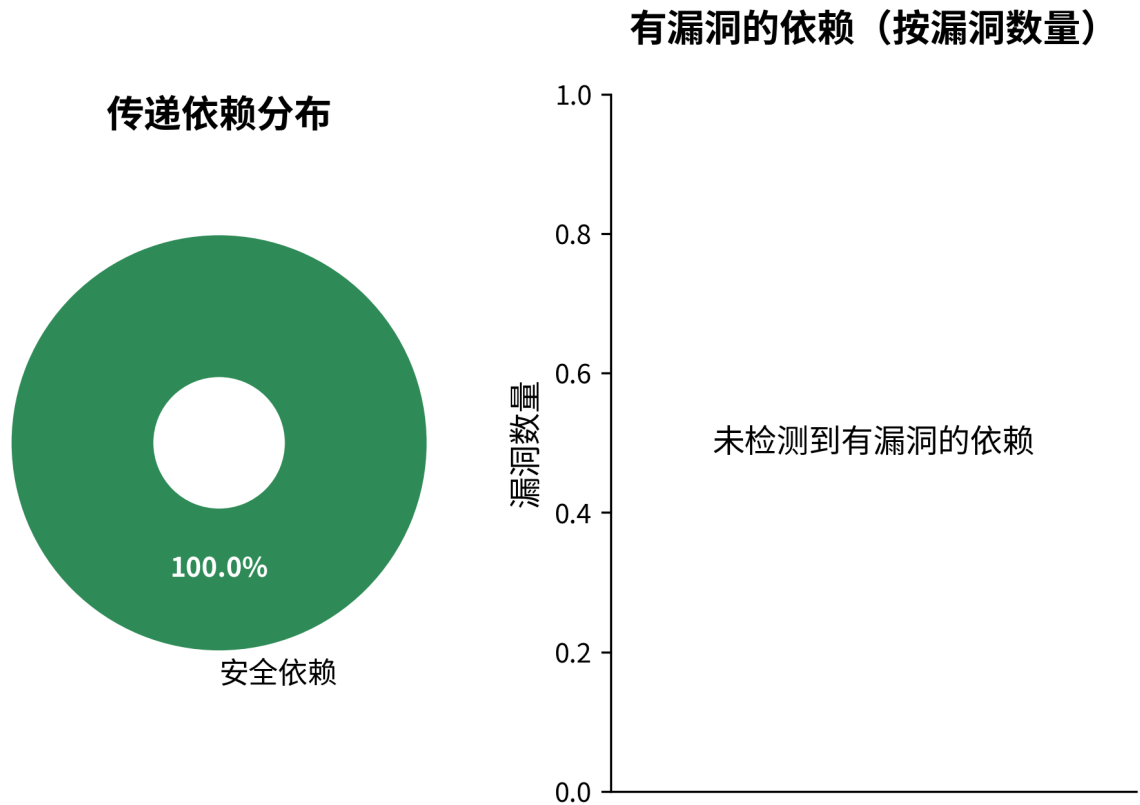


图 2.2 项目依赖构成与安全漏洞分析

- 危险的函数调用（如 `eval()`, `subprocess.Popen` 的 Shell 注入风险）。
- 不安全的网络绑定（如绑定到 0.0.0.0）。

2.3.2 风险评估

扫描结果被分类为 High, Medium, Low 三个风险等级。图 2.3 直观展示了各类风险的分布情况，这为安全修复优先级提供了数据支持。

2.4 代码质量与动态分析

除了上述特定的分析外,工具还整合了 `pylint` 进行代码风格检查,以及 `pytest-cov` 进行动态测试覆盖率分析。

- **PyLint 质量分布:** 图 2.4 展示了代码中各类问题的比例,包括规范 (Convention)、重构 (Refactor)、警告 (Warning) 和错误 (Error)。

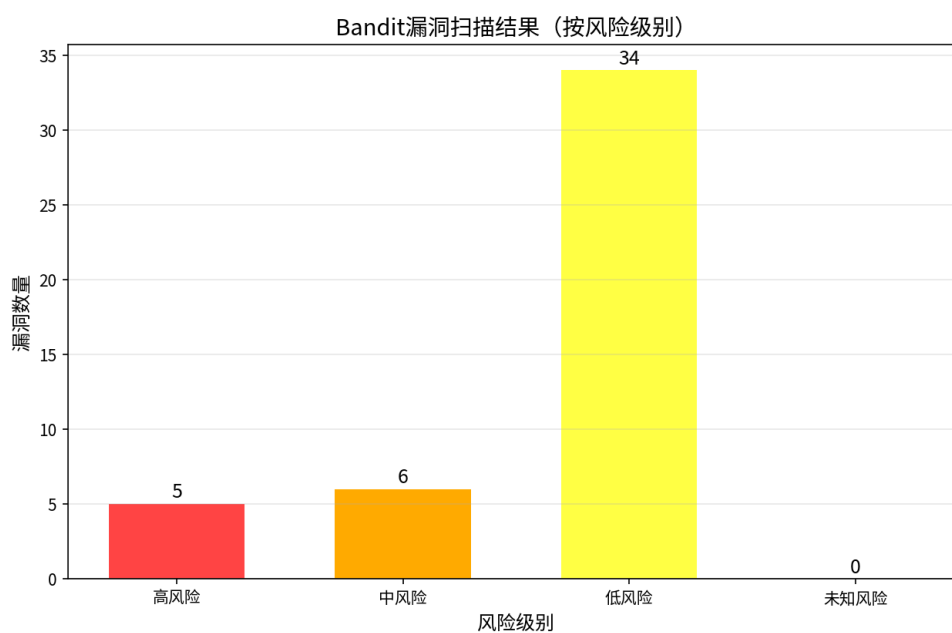


图 2.3 Bandit 静态漏洞扫描结果分布

- **动态测试覆盖率：**analyze.py 模块尝试运行项目中的单元测试，并生成覆盖率报告（'coverage_report.txt'），以评估测试用例对代码逻辑的覆盖程度。

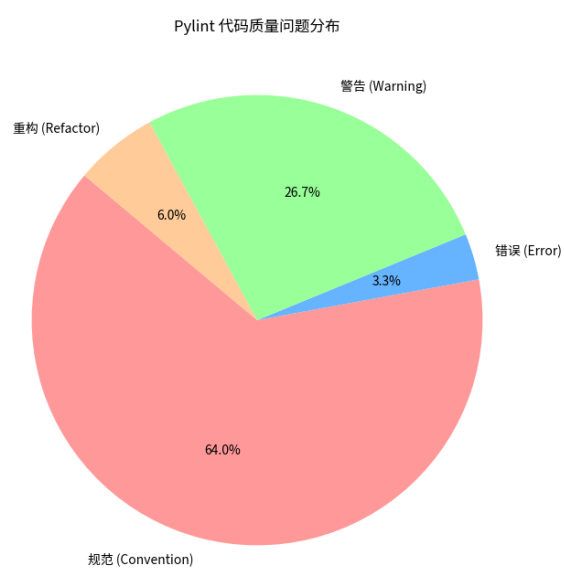


图 2.4 Pylint 代码质量问题分布

3 github 仓库分析

本章介绍基于 GitHub API 的仓库数据挖掘与分析。依托 PyGithub 库，工具实现了对 Pull Request 协作流程、Issue 问题追踪以及 GitHub Actions CI/CD 工作流的深度透视。

3.1 GitHub PR (Pull Request) 分析

Pull Request 是开源项目协作的核心载体，其处理效率直接反映了社区的活跃度与维护者的响应能力。`pr_analysis.py` 模块主要实现了以下维度的分析：

3.1.1 生命周期与状态统计

工具抓取了仓库的全量 PR 数据，计算了 PR 的合并率 (Merged Rate) 与拒绝率 (Rejected Rate)。如图 3.1 所示，左侧饼图直观展示了 PR 的最终流向状态。

3.1.2 僵尸 PR (Zombie PRs) 识别

为了帮助维护者清理积压任务，模块通过设定时间阈值（默认为 7 天无活动），识别出长期未处理的“Zombie PRs”。这能够提醒社区管理者关注那些被遗忘的贡献。

3.1.3 审查效率评估

分析器计算了每个 PR 从创建到关闭/合并的平均耗时（平均生命周期），以及平均 Review 评论数。这些指标量化了代码审查流程的效率与质量。

3.2 GitHub Issue 安全分析

Issue 不仅是 Bug 报告的入口，也是发现潜在安全风险的重要渠道。`github_issue_analyzer.py` 模块专注于从海量 Issue 中挖掘安全线索。

3.2.1 安全关键词挖掘

该分析器定义了一套覆盖广泛的安全关键词词典（如 *xss*, *injection*, *vulnerability*, *CVE*, *rce*, 溢出等）。它自动遍历 Issue 的标题、正文及标签，筛选出可能涉及安全漏洞的条目。

3.2.2 趋势与分类统计

图 3.2 展示了分析结果。

- **趋势分析：**右上角的折线图展示了近期 Issue 的提交与关闭趋势，反映了项目的问题吞吐量。

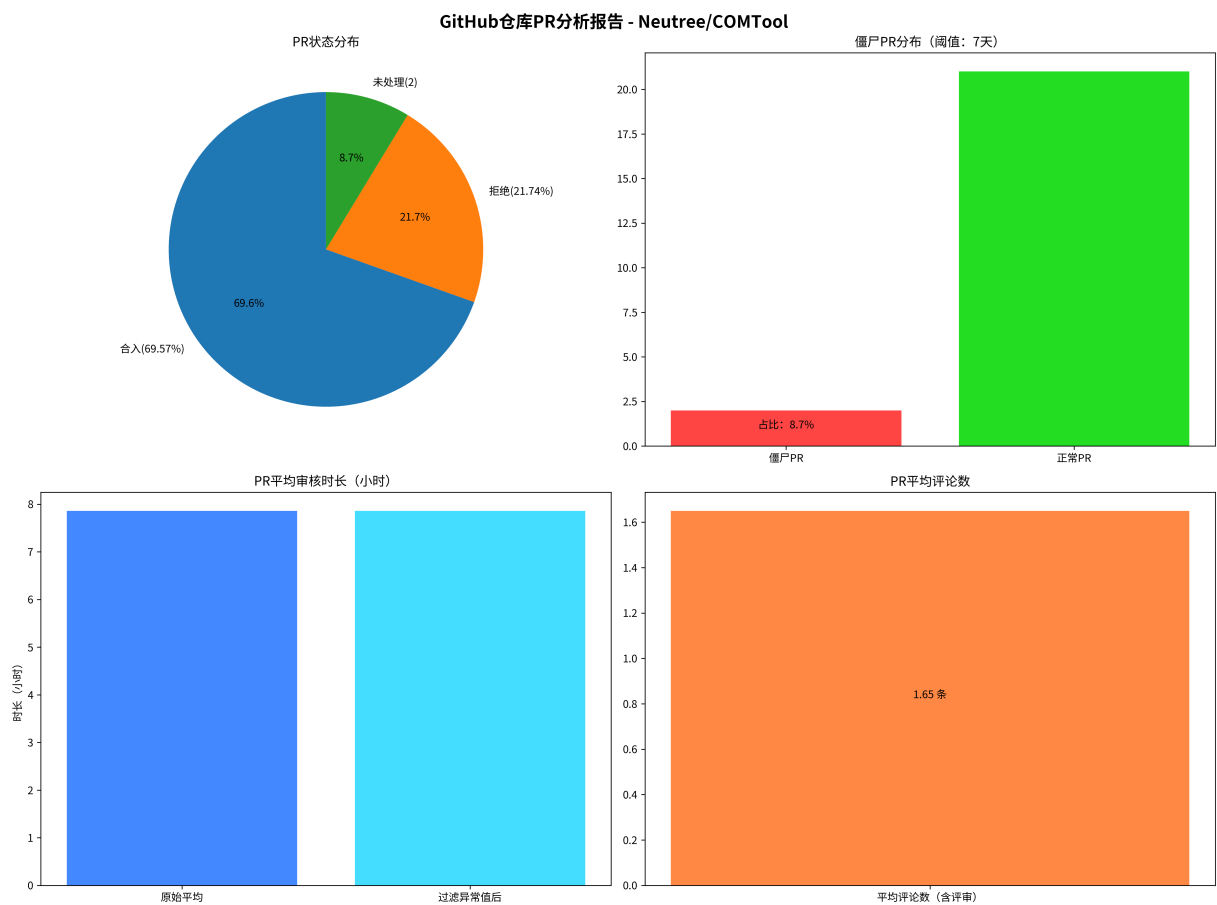


图 3.1 Pull Request 状态分布与审查效率仪表盘

- **安全占比:** 右下角的饼图则突出了安全相关 Issue 在整体问题中的占比，帮助安全团队快速评估项目的风险暴露面。

3.3 GitHub Actions workflow分析

GitHub Actions 是现代项目自动化的基石。github_actions_analyzer.py 模块旨在评估 CI/CD 流程的安全与效率。

3.3.1 workflow安全性检测

工具内置了基于正则表达式的风险模式库，对 ‘github/workflows’ 下的 YAML 文件进行静态扫描。主要检测：

- **高危模式:** 如使用 `curl | bash` 直接执行远程脚本，或 `pull_request_target` 事件的不当使用。
- **供应链风险:** 如引用第三方 Action 时使用可变的 version tag 而非不可变的 commit

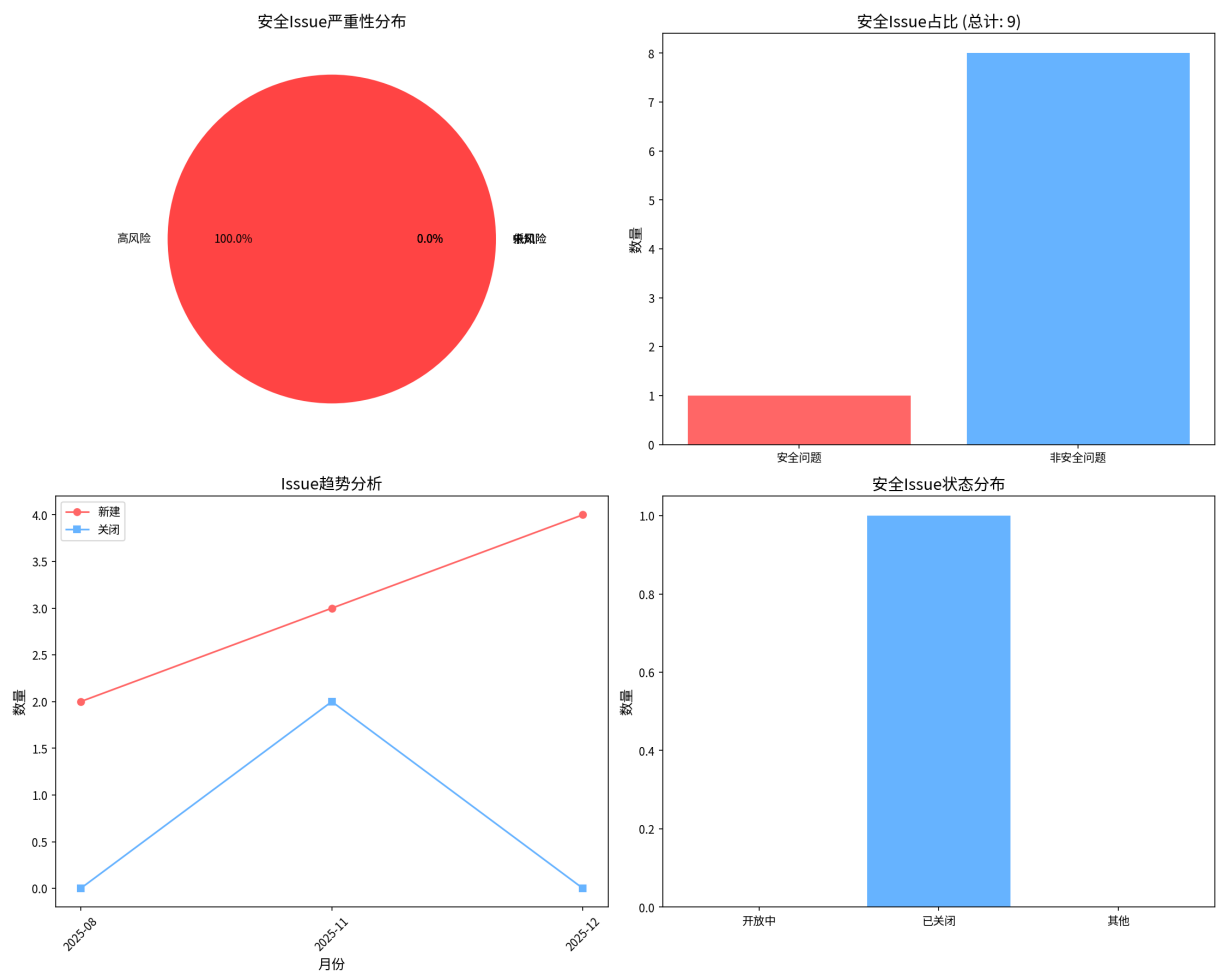


图 3.2 Issue 趋势与安全相关性分析

hash。

- **权限配置：**检查是否存在过宽的权限授予（如默认 write-all）。

3.3.2 运行效率分析

通过分析最近的工作流运行记录 (Workflow Runs)，统计了运行成功率与平均耗时。图 3.3 综合展示了工作流的健康状况（成功/失败比例）以及各项安全指标的检测结果。

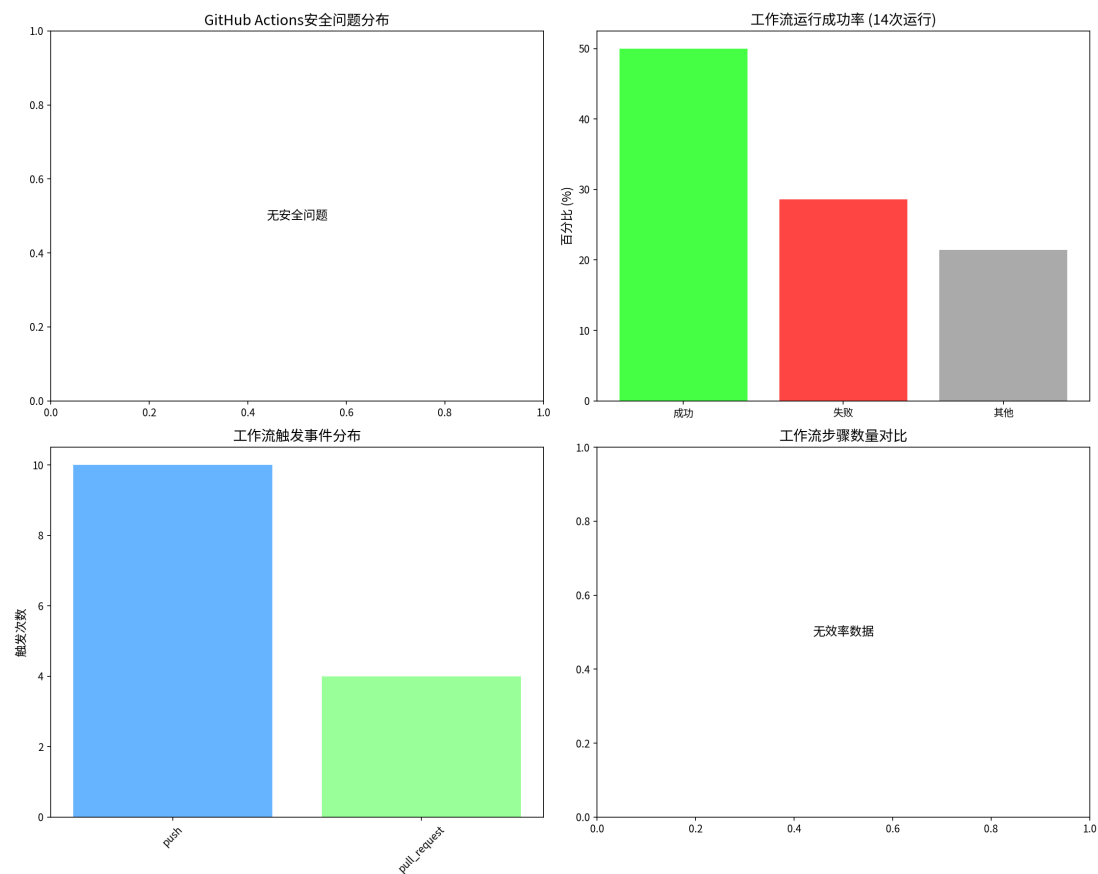


图 3.3 GitHub Actions 工作流安全与效率分析

结 论

本文设计并实现了一套多维度的软件仓库分析工具，通过整合 Git 版本控制数据、代码静态分析以及 GitHub 平台元数据，为软件项目的质量评估与演进分析提供了全面的解决方案。

主要工作总结如下：

- (1) **可视化 Git 历史分析**：通过 `gitgraph.js` 和多维度统计图表，直观展示了项目的开发演进路线、团队贡献分布及开发节奏，帮助管理者快速把握项目现状。
- (2) **深度代码质量评估**：集成了圈复杂度计算、依赖树分析及漏洞扫描功能。能够自动识别高复杂度代码块、过时或危险的第三方依赖，以及潜在的代码安全漏洞，为代码重构和安全加固提供精准指引。
- (3) **GitHub 协作流程洞察**：实现了对 Issue, Pull Request 及 GitHub Actions 的自动化分析。不仅量化了社区的响应效率（如 PR 处理周期），还通过关键词挖掘识别了潜在的安全风险，并评估了 CI/CD 流程的效率与安全性。

本工具采用模块化设计，易于扩展。未来的工作将集中在支持更多编程语言的分析、引入 AI 模型进行更深层次的代码语义分析，以及开发实时的仪表盘监控系统，进一步提升工具的实用性与智能化水平。

附录 A 附录内容名称